

Aula 1 - Javascript

Programação em Dispositivos Móveis

Carmen Silva

*Universidade Técnica do Atlântico
Instituto de Engenharia e Ciências do Mar*

24 de março de 2026

Programação

1 JavaScript

JavaScript

JavaScript

- Com o tempo, passou a ser utilizada fora do navegador, através de ambientes como o Node.js.
- Tornou-se uma linguagem versátil para desenvolvimento full-stack (frontend e backend).
- É essencial para aprender React, uma biblioteca JavaScript para construção de interfaces de utilizador.
- O React utiliza conceitos fundamentais do JavaScript como funções, objetos, arrays, módulos e programação assíncrona.
- Aprender JavaScript é o primeiro passo para desenvolver aplicações web e móveis com React.

JavaScript

- Tutorial de JavaScript - W3Schools

JavaScript: Variáveis

- Uma variável em JavaScript é um espaço na memória usado para guardar dados, funcionando como um nome que representa um valor (número, texto, lista, objeto, etc.), sendo essencial para construir programas e aplicações, especialmente com bibliotecas como o React.
- Em JavaScript, existem três formas principais de declarar variáveis:
 - **let** → usada quando o valor pode mudar
 - **const** → usada quando o valor não deve mudar
 - **var** → forma antiga (evitar sempre que possível)

```
1 let idade = 38;  
2 const pi = 3.14;  
3 var nome = "Gong Yoo";
```

JavaScript: Reatribuição de Valores

- Variáveis declaradas com `let` podem mudar de valor, enquanto `const` não permite reatribuição.

```
1 let idade = 18;  
2 idade = 38; // valido  
3  
4 const pi = 3.14;  
5 pi = 3.1416; // erro
```

JavaScript: Tipos de Dados e Escopo das Variáveis

- JavaScript é uma linguagem de tipagem dinâmica, ou seja, não é necessário declarar o tipo da variável.

```
1 let valor = 10;      // numero
2 valor = "dez";      // string
```

- O escopo define onde a variável pode ser usada:
 - let e const → escopo de bloco {}
 - var → escopo de função

```
1 if (true) {
2     let x = 10;
3 }
4 // x nao existe aqui
5
6 if (true) {
7     var y = 20;
8 }
9 // y ainda existe aqui
```

JavaScript: Arrow Functions

- Arrow functions são uma forma concisa de escrever funções em JavaScript, introduzidas no ECMAScript 6 (ES6).
- Elas permitem criar funções de forma mais curta e clara, especialmente úteis para funções anónimas ou funções passadas como argumentos para outros métodos, como em arrays ou em bibliotecas como React.

JavaScript: Arrow Functions

- A sintaxe básica de uma arrow function é:

```
1 const nomeFuncao = (param1, param2) => {  
2     // corpo da funcao  
3     return param1 + param2;  
4 };
```

- `const nomeFuncao` → define a função como constante
- `(param1, param2)` → parâmetros da função
- `=>` → indica que é uma arrow function
- `{}` → delimita o corpo da função

JavaScript: Arrow Function

```
1 function myFunction(a,b){  
2     return a+b;  
3 }  
4  
5 const myArrowFunction = (a,b) => {  
6     return a+b;  
7 };  
8  
9 const myArrowFunction2 = (a,b) => a+b;  
10  
11 const random = () => 10;  
12  
13 console.log(myFunction(2,3), myArrowFunction(5,6),  
             myArrowFunction2(20,2), random());
```

JavaScript: Objetos

- Em JavaScript, um objeto é uma coleção de propriedades (dados) e métodos (funções associadas a esses dados).
- Objetos permitem organizar informações de forma estruturada e são fundamentais em bibliotecas como React, pois componentes muitas vezes trabalham com objetos para gerir estado e propriedades.

JavaScript: Criação de Objetos

- Existem várias formas de criar objetos:
 - Utilizando notação literal:

```
1 const pessoa = {  
2     nome: "Maria",  
3     idade: 60  
4 };
```

- Utilizando o construtor Object:

```
1 const pessoa = new Object();  
2 pessoa.nome = "Manuel";  
3 pessoa.idade = 66;
```

JavaScript: Propriedades de Objetos

- As propriedades são variáveis associadas a um objeto, que armazenam valores ou outros objetos.

```
1  const carro = {  
2      marca: "Toyota",  
3      modelo: "Corolla",  
4      ano: 2020  
5  };  
6  console.log(carro.marca); // "Toyota"
```

- Para aceder às propriedades: **objeto.propriedade** ou **objeto["propriedade"]**

JavaScript: Métodos de Objetos

- Os métodos são funções dentro de objetos, que podem manipular as propriedades ou executar ações.

```
1 const pessoa = {  
2   nome: "Riccardo",  
3   idade: 15,  
4   cumprimentar: function() {  
5       console.log('Ola, o meu nome e ${this.  
6       nome}');  
7   }  
8 };  
9 pessoa.cumprimentar(); // "OI , o meu nome e  
10 Riccardo"
```

- **this** refere-se ao próprio objeto

JavaScript: Métodos

- Em ES6, métodos podem ser declarados de forma mais concisa:

```
1 const pessoa = {  
2     nome: "Maria",  
3     cumprimentar() {  
4         console.log('Ola, o meu nome e ${this.  
5         nome}');  
6     }  
7 };  
8 pessoa.cumprimentar(); // "Ola, o meu nome e  
9     Maria"
```

JavaScript: Adicionar ou Remover Propriedades e Métodos

- Adicionar: `objeto.novaProp = valor`
- Remover: `delete objeto.prop`

```
1 const livro = { titulo: "JS Avancado" };  
2 livro.autor = "David"; // adicionar  
3 delete livro.titulo;    // remover
```

JavaScript: Iterar sobre objetos

- É possível percorrer as propriedades de um objeto utilizando `for...in`

```
1 const pessoa = { nome: "Maria", idade: 60 };  
2 for (let chave in pessoa) {  
3     console.log(chave + ": " + pessoa[chave]);  
4 }  
5 // nome: Maria  
6 // idade: 60
```

JavaScript: Array

- Em JavaScript, um array é uma estrutura de dados que permite armazenar múltiplos valores numa única variável, mantendo a ordem desses valores.
- Arrays são fundamentais para organizar listas de elementos e são muito usados em aplicações com React, por exemplo para gerir listas de componentes ou dados do utilizador.
- arrays podem ser criados de várias formas:

- Utilizando notação literal:

```
1 const numeros = [1, 2, 3, 4, 5];
```

- Utilizando o construtor Array:

```
1 const frutas = new Array("Ma", "Banana",  
    "Laranja");
```

JavaScript: Aceder e Modificar Elementos de arrays

- Os elementos de um array são indexados a partir do 0.

```
1 const cores = ["vermelho", "verde", "azul"];  
2 console.log(cores[0]); // "vermelho"  
3 cores[1] = "amarelo";  
4 console.log(cores); // ["vermelho", "amarelo", "  
    azul"]
```

JavaScript: Métodos Básicos de arrays

- **push()** → adiciona um elemento no final
- **pop()** → remove o último elemento
- **shift()** → remove o primeiro elemento
- **unshift()** → adiciona um elemento no início

```
1 const numeros = [1, 2, 3];  
2 numeros.push(4);      // [1, 2, 3, 4]  
3 numeros.pop();        // [1, 2, 3]  
4 numeros.shift();      // [2, 3]  
5 numeros.unshift(0);   // [0, 2, 3]
```

JavaScript: Métodos de Iteração e Transformação de arrays

- **forEach()** → executa uma função para cada elemento
- **map()** → transforma cada elemento e retorna um novo array
- **filter()** → retorna um novo array com elementos que passam na condição
- **reduce()** → reduz o array a um único valor

```
1 const numeros = [1, 2, 3, 4];  
2 numeros.forEach(n => console.log(n * 2)); // 2,  
   4, 6, 8  
3 const quadrados = numeros.map(n => n * n); //  
   [1, 4, 9, 16]  
4 const pares = numeros.filter(n => n % 2 === 0);  
   // [2, 4]  
5 const soma = numeros.reduce((total, n) => total  
   + n, 0); // 10
```

JavaScript: Métodos Procurar e Encontrar Elementos

- **find()** → retorna o primeiro elemento que satisfaz a condição
- **findIndex()** → retorna o índice do primeiro elemento que satisfaz a condição
- **includes()** → verifica se um elemento existe no array

```
1 const frutas = ["Ma   ", "Banana", "Laranja "];  
2 console.log(frutas.find(f => f.startsWith("B")))  
   ; // "Banana"  
3 console.log(frutas.findIndex(f => f === "Laranja"  
   )); // 2  
4 console.log(frutas.includes("Ma   ")); // true
```

JavaScript: Métodos para Procurar e Encontrar Elementos

- **sort()** → ordena os elementos
- **reverse()** → inverte a ordem

```
1 const numeros = [3, 1, 4, 2];  
2 numeros.sort(); // [1, 2, 3, 4]  
3 numeros.reverse(); // [4, 3, 2, 1]
```

